

Chunking Strategies

Dove tagli conta

fixed, recursive, semantic

Modulo M2 • Lezione 22



Sono arrivati i documenti lunghi

Tre documenti veri di Lumen, più lunghi di quelli visti finora.



`guida-smart-working.txt`

La policy HR: giornate da remoto, la postazione di casa, contributi e regole.

5.100 caratteri



`policy-resi-garanzia.txt`

Recesso, rimborsi, garanzia 5+2 anni, l'estensione SRV-O3, le eccezioni.

5.396 caratteri



`manuale-lumadesk.txt`

Il manuale della LD-200: montaggio, centralina, memorie, codici di errore.

5.681 caratteri

Sono **già puliti** (quella fatica l'avete fatta a L17-L18) — ma **16.177 caratteri in 3 vettori**? Un testo così non sta sensatamente in un embedding solo. Vediamo perché.



La regia della serata

Un solo file per tutta la serata: `working/chunk.py` .

	Blocco	Dove si lavora	Tempo
	Teoria: perché si spezza, le tre strategie, i costi	slide	40'
	Live coding: <code>chunk_fixed</code> e <code>chunk_recursive</code> (le scrivete) + demo <code>chunk_semantic</code> (si guarda)	<code>working/chunk.py</code>	45'
	Esperimenti: le manopole, le tre query, la tabella	<code>working/chunk.py</code>	30'
	Wrap-up: reveal + ponte a giovedì	slide	5'

 = scrive il docente, voi replicate (o guardate)  = scrivete voi, docente in affiancamento

Chiave OpenAI: quella di L13–L21 (persa? alzate la mano). La collection si **ricrea da zero** a ogni giro: cambi un'impostazione, rifai l'indice.



1. Perché si spezza



La prova: le query sui documenti interi

Indicizziamo i 3 documenti così come sono — un vettore l'uno — e facciamo le tre domande della serata:

Query	Documento in testa	Score	E il 2°?
«Quanti giorni per il reso comprato nei saldi?»	policy-resi-garanzia	0.466	0.340
«Quanto vale il contributo postazione, entro quando?»	guida-smart-working	0.421	0.328
«La centralina mostra E07: cosa devo fare?»	manuale-lumadesk	0.380	0.330

Score piatti

Il vettore di un documento intero è la **media di dieci argomenti**: non somiglia davvero a nessuna domanda. I secondi posti stanno a un soffio — il metro **non discrimina**.

«Risultato» = 5.000+ caratteri

Il documento giusto **non è la risposta**: è un compito a casa («cercatela voi lì dentro»). E giovedì questi risultati andranno **dentro un prompt**.

Trovare il **documento** giusto non basta: il retrieval deve trovare il **pezzo** giusto.



Le due ragioni (e una definizione)

Due problemi diversi, una sola causa: il documento intero è l'unità sbagliata.

Context window (e conto in token)

Giovedì i risultati finiscono **dentro un prompt**: un documento intero non ci sta — e quando ci sta, lo **paghi tutto** (L14) per usarne tre righe.

Granularità del retrieval

Un embedding per 10 argomenti = significato **annacquato**: la media di tutto non somiglia a niente. L'avete appena visto: score piatti, secondi posti attaccati.

Il chunk è l'unità del retrieval

Si spezza **prima** di embeddare · si indicizza il **pezzo** · si recupera il **pezzo**. Tutta la domanda della serata è: **DOVE si taglia?**

E il non-chunking? È solo un chunking con **size infinito** — infatti stasera si prova con la stessa manopola.

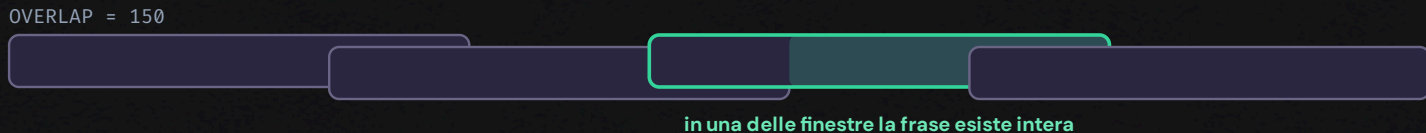


2. Le tre strategie



Fixed-size: ogni N caratteri, taglio

La più semplice: blocchi di size caratteri — anche a metà parola, anche a metà frase. E l'overlap: ogni blocco riparte un po' indietro.



Pro: prevedibile

Sai quanti chunk escono e di che taglia — sempre, su qualsiasi testo.

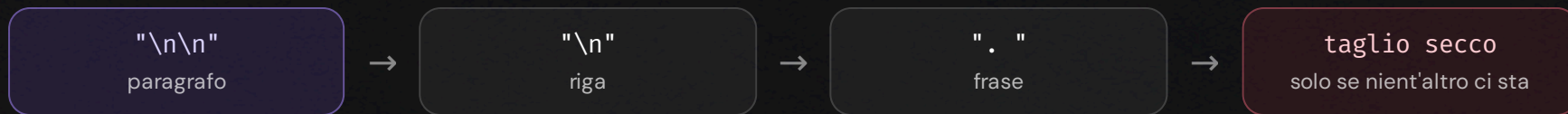
L'overlap è una rete di sicurezza COMPRATA

Ogni confine esiste anche ricucito — ma ogni carattere ripetuto è un token ripagato. Il conto tra poco.



Recursive: il taglio rispetta il testo

Si prova il separatore più forte, e si scende solo se serve: il taglio cade sull'ultimo separatore che ci sta nel size, poi si riparte sul resto — per questo si chiama *ricorsivo*.



Pro: mai una parola rotta

I tagli cadono su titoli, paragrafi e frasi: chunk che **sembrano scritti apposta**.

Contro: si fida della FORMA

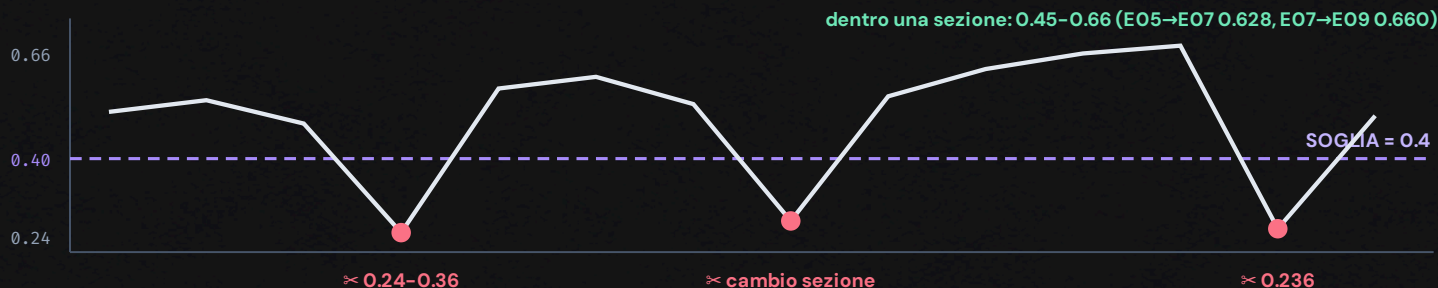
Se chi scrive va a capo a metà **ragionamento**, il taglio lo segue nel burrone. Il testo non sempre va a capo dove cambia il discorso.

Nei framework la incontrerete come *RecursiveCharacterTextSplitter* — stasera la scrivete in **dieci righe**.



Semantic: il taglio segue il discorso

Gli embedding sono strumenti vostri da L19: embedding di ogni **frase**, similarità tra frasi **adiacenti** — dove crolla, il discorso è cambiato → taglio. Misurato sul manuale:



I tagli si trovano da soli

Con soglia **0.4** i tagli cadono sui **confini di sezione** — nessuno gli ha detto dove sono i titoli.

Si paga due volte

Prima le **frasi** (per decidere i tagli), poi i **chunk** (per l'indice): il chunking stesso va a consumo.



La stessa pagina, spezzata nei tre modi

La sezione «saldi» della policy, tre geometrie. Il retrieval le vedrà **diversissime**:

fixed 500/0

...rimborso proporzionale ai prodotti rientrati

PRODOTTI IN PROMOZIONE E SALDI · ...la finestra di reso ✕

✕ scende a 14 giorni... buono acquisto...

blocchi identici, frase **spezzata**

recursive 500

RIMBORSI · il rimborso parte quando...

PRODOTTI IN PROMOZIONE E SALDI · ...
scende a 14 giorni... buono acquisto ✓

GARANZIA LEGALE E COMMERCIALE · ...

tagli sui separatori, frase **intera**

semantic 0.4

...tutto il blocco «rimborsi», insieme

il blocco «saldi», dal titolo alla fine ✓

il blocco «garanzia», insieme

un chunk = **un argomento**

Sull'intero corpus: **34 · 45 · 77** chunk — stessa pagina, tre geometrie.

misurato sui 3 documenti



I trade-off

Due manopole principali, quattro estremi — e nessuna casella è «quella giusta»:

`CHUNK_SIZE` piccolo

Granularità alta, score «puliti» — ma contesto **povero** e tanti pezzi da gestire.

`CHUNK_SIZE` grande

Contesto ricco — ma significato **annacquato**: il documento intero è il caso limite.

`OVERLAP` alto

Confini **ricuciti** (nessuna frase orfana) — ma ridondanza **pagata** in token.

`OVERLAP` zero

Gratis — ma ogni confine è un **punto cieco**: quello che cade lì, si spezza.

E per semantic? La manopola è la **SOGLIA**: alta = tagli ovunque, bassa = chunk lunghi.

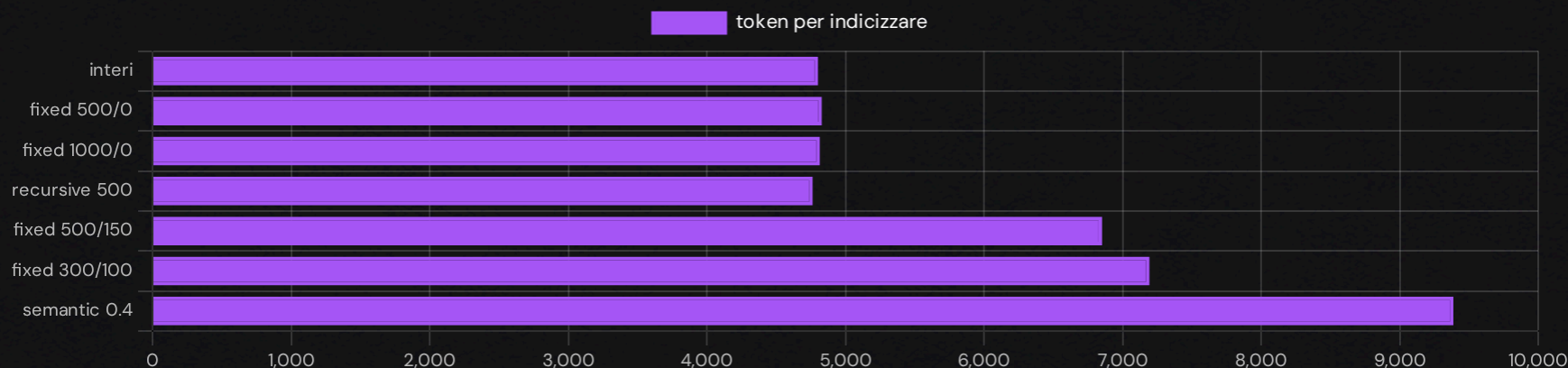
Dipende da **documenti** e **domande** — e stasera lo misurate con le vostre mani.



Quanto costa

Il conto dell'indicizzazione dei 3 documenti, configurazione per configurazione:

Token per indicizzare i 3 documenti



Spezzare è gratis (stessi caratteri, stesso conto) · **l'overlap si paga** (+42% a 500/150) · **semantic raddoppia** (le frasi si pagano). Qui sono frazioni di centesimo — a milioni di documenti questo rapporto è il budget.



Iniziamo: l'ambiente

Quattro righe e siete operativi (passo-passo anche in `scripts/CONSEGNA.md`):

```
cd scripts
python3 -m venv .venv && source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env # dentro: la STESSA chiave OpenAI di L13-L21
```

Niente corpus da copiare stasera

I documenti sono in chiaro in `dataset/`: gli embedding si calcolano a ogni giro e la collection `lumen_chunks` si ricrea da zero — è il punto della serata.

aikit/ è cresciuto di un modulo

`vectorstore.py` — il VOSTRO, scritto sabato, promosso nel toolkit. Si importa, non si riscrive.



🎙 Live coding



Hint: osservate e replicate

working/chunk.py · 45 minuti

si torna alle slide
dopo la demo

TODO 1 · ~15' · si replica

`chunk_fixed`

Un `while`, una fetta di stringa, l'inizio che
avanza di `size - overlap`.

TODO 2 · ~15' · si replica

`chunk_recursive`

`rfind` dell'ultimo separatore nella finestra,
il pezzo, il resto, la **ricorsione**.

DEMO · ~10' · si guarda

`chunk_semantic`

È **già nel file**: la leggiamo insieme e la lancio
io. Voi la **userete** negli esperimenti — capirla
sì, riscriverla no.

Dopo ogni funzione: si lancia su un documento vero

Lo script stampa i **tagli** (dove cadono, quanti chunk, di che taglia) e le **tre query** con il primo risultato per intero. Guardateli: sono già i primi dati della vostra tabella.



Recap: le tre strategie

AL RIENTRO

Quello che gli occhi hanno appena visto:

fixed taglia alla cieca

A 500/0 le tre query tornano **tutte tagliate**: il chunk dei saldi si interrompe su «...la finestra di reso» — il numero è nel chunk dopo.

recursive rispetta i separatori

Saldi 0.695 ed E07 0.735 tornano **interi in testa...** ma il CONTRIBUTO resta tagliato anche coi tagli educati.

semantic si orienta da sola

Tagli sui **confini di sezione** senza conoscerli — e a schermo la riga `[openai]` è passata **due volte**: si paga.

Il mistero: perché il contributo resiste?

La risposta è negli **esperimenti**. Indizio: guardate **dove finisce un paragrafo e dove comincia l'altro**, nella guida smart working.



Esperimenti



TOCCA A VOI — muovete le manopole

working/chunk.py · 30 minuti · si toccano SOLO le 4 manopole in testa al file

si torna alle slide
al mio segnale, per il reveal

Il giro

Cambia **UNA** manopola → rilancia → leggi il **1° risultato per intero** → annota una riga nella **tabella in fondo al file**: ✓ integra in testa · ✕ tagliata · ~ sepolta.

da provare

500/0 ✓ (fatta) · overlap 150 · size 1000 · 300/100 · recursive · semantic 0.4 e 0.5

Domanda guida: quale config porta a casa **TUTTE e tre** le query? Ce n'è una sola?

Completato quando:

Ne avete trovata **almeno una** — e sapete dire **quale manopola** ha fatto la differenza.

Occhio al contatore

La riga [openai] ... \$ passa a ogni giro: **alcune config costano più di altre** — annotate anche quello.



Il reveal: la tabella della serata

AL RIENTRO

Compilata in preparazione — le vostre dovrebbero somigliarle:

Config	chunk	saldi	contributo	E07
documenti interi	3	~	~	~
fixed 500/0	34	✗	✗	✗
fixed 500/150	48	✓ 0.684	✗ *	✗
fixed 1000/0	18	✗	~	~
fixed 300/100	82	✓ 0.646	✗	✗
recursive 500	45	✓ 0.695	✗	✓ 0.735
semantic 0.4	77	✓ 0.699	✓ 0.527	✓ 0.665
semantic 0.5	119	✓	✗	✓

(*) la storia più fine della serata: a 500/150 il chunk ricucito dall'overlap **ESISTE** (importo E scadenza insieme) ma resta **terzo** — l'overlap ricuce il testo, non garantisce il ranking. Il contributo era **a cavallo di due paragrafi**: recursive taglia proprio lì; semantic vede che il discorso **continua** (le frasi si somigliano) e non taglia.



Si sceglie sperimentando

Il metodo, prima ancora del codice:

01

Nessuna strategia domina

Stasera ognuna ha vinto e perso da qualche parte: fixed la più economica, recursive la più pulita, semantic la più precisa (e cara).

02

La config giusta esiste — QUI

semantic 0.4 vince su questo corpus e queste domande. Su contratti, codice o chat la tabella si rifà da capo.

03

Il giro è vostro

Cambia una manopola → rilancia → leggi → annota. Vale per il chunking come per tutto il tuning che arriverà.

Il chunking è la leva più a monte

Tutto quello che viene dopo — ricerca, prompt, risposta — **eredita i vostri tagli**. Un taglio sbagliato non si recupera a valle.



Giovedì: il primo RAG completo

Sul disco vi resta `lumen_chunks` indicizzata con la vostra config migliore — **non buttatela**. Giovedì il cerchio si chiude:



query → search sulla collection → i chunk DENTRO un prompt → risposta con citazioni

Un mese di lavoro diventa il primo **RAG completo** del corso: l'LLM risponde *citando i documenti di Lumen*.



Grazie

(L23): Prompt Engineering Avanzato — i vostri chunk dentro un prompt: il primo RAG completo del corso.

